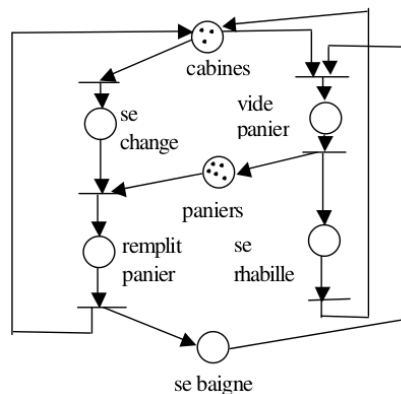


	Programmation système	2025/2026
	Projet N°3	
	Processus	

Objectifs :

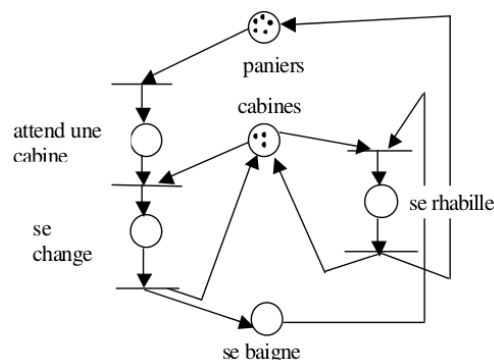
- Maîtriser la communication, l'ordonnancement et la synchronisation entre processus

On veut simuler le fonctionnement d'une piscine en considérant chaque baigneur qui se présente comme un processus accomplissant un certain nombre de tâches (ex. prendre un panier, se baigner, choisir une cabine (rhabillage), rendre le panier), à l'aide de ressources (paniers, cabines) qu'il doit partager avec les autres baigneurs. Si ces ressources ne sont pas disponibles, il est bloqué, il doit attendre. On dispose de NP paniers et NC cabines ($NC < NP$). Prenons par exemple le cas où il y a trois cabines et 5 paniers. La Figure suivante montre une modélisation d'un premier scénario : choisir une cabine, prendre un panier, se baigner, choisir une cabine (rhabillage), rendre le panier.



Représentation par un réseau de Petri : Scénario 1 (sans blocage)

Cependant, le blocage se produit quand il y a 5 baigneurs (plus de paniers), 3 clients en attente de panier dans les cabines et plus de cabines disponibles pour qu'un baigneur libère un panier. La solution sans blocage consiste en entrée de la piscine à prendre un panier avant de chercher une cabine, comme illustrée dans la Figure suivante.



Représentation par un réseau de Petri : Scénario 2 (sans blocage)

Écrire le processus baigneur en ajoutant les synchronisations à l'aide de sémaphores. Nous pouvons assimiler les baigneurs à des processus concurrents; les cabines et les paniers sont les ressources partagées.

```
void nageur()
{ se_déshabiller(); /* nécessite un panier et une cabine */
  Se baigner ();
  se_rhabiller(); /* nécessite une cabine */
}
```

Dates de rendu (plateforme):

- Document final (max 5 pages) + sources + jeux de tests
- Le rapport, les sources et les jeux de tests seront rendus dans une archive (nom-prenom-Projet3.zip/gz).